



Gerenciador de Tarefas

Repositório no GitHub: [andrecesarvieira/tarefas](https://github.com/andrecesarvieira/tarefas)

Índice

1. Descrição do Projeto
 2. Problema a Resolver
 3. Solução Proposta
 4. Escopo do Projeto
 5. Usuários do Sistema
 6. Ubiquitous Language
 7. Regras de Negócio
 8. Testes Unitários
 9. Diagrama da Arquitetura DDD
 10. Critérios de Avaliação (Rubrica)
-

Descrição do Projeto

O **Gerenciador de Tarefas** é uma aplicação que permite que usuários criem, gerenciem e controlem suas tarefas diárias. A aplicação fornece um sistema robusto para organizar trabalhos a serem realizados, atribuindo prazos e níveis de prioridade a cada tarefa, garantindo a integridade dos dados e o cumprimento das regras de negócio.

A solução foi desenvolvida utilizando **Domain-Driven Design (DDD)** como arquitetura principal, aplicando conceitos de **Orientação a Objetos**, **SOLID**, **GRASP** e **testes unitários** para garantir um código limpo, testável e facilmente extensível.

Problema a Resolver

Problema identificado:

Usuários precisam de uma forma simples, confiável e bem estruturada de gerenciar suas tarefas diárias. Frequentemente enfrentam:

- Dificuldade em organizar tarefas sem perder informações
- Necessidade de atribuir prazos e prioridades

- Risco de alterar tarefas já concluídas (causando inconsistências)
- Falta de um sistema que valide dados de entrada automaticamente
- Dificuldade em rastrear tarefas próximas do vencimento

Exemplo de cenário problemático:

Um usuário tenta:

1. Criar uma tarefa sem título (não deveria permitir)
2. Atribuir um prazo no passado (tarefa impossível de cumprir)
3. Concluir uma tarefa que já foi concluída (duplicação)

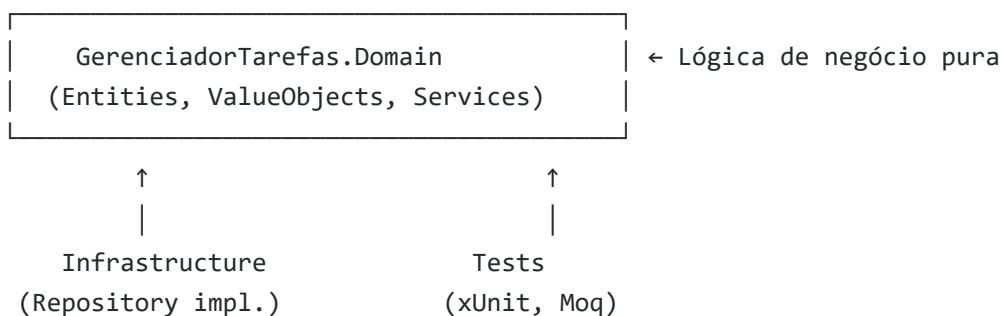
Sem validações, o sistema fica inconsistente.

Solução Proposta

Desenvolver uma aplicação baseada em **Domain-Driven Design** que modele o domínio de tarefas de forma clara e centralizada, garantindo que:

- ☒ **As regras de negócio estejam no coração da aplicação** (camada de domínio)
- ☒ **Dados sejam validados no ponto de entrada** (construtores, value objects)
- ☒ **Operações complexas sejam coordenadas por domain services**
- ☒ **Código seja testável com cobertura >80%**
- ☒ **Princípios SOLID e GRASP sejam aplicados**

Arquitetura:



Escopo do Projeto

O que está DENTRO da aplicação

- ☒ **Criação de tarefas** com título, descrição, prazo e prioridade
- ☒ **Validação de dados de entrada:**
 - Título obrigatório e não vazio
 - Prazo não pode ser no passado
 - Usuário ID obrigatório
- ☒ **Marcação de tarefas como concluídas**
- ☒ **Recuperação de tarefas por ID**
- ☒ **Atualização de tarefas existentes**
- ☒ **Exclusão de tarefas**
- ☒ **Notificações sobre tarefas próximas do vencimento** (via Anti-Corruption Layer)
- ☒ **Persistência de dados** (interface via Repository Pattern)
- ☒ **Testes unitários** com cobertura >80%

O que está FORA da aplicação

- ☒ **Interface gráfica (UI)** - apenas lógica de domínio
 - ☒ **Autenticação e autorização** de usuários
 - ☒ **Envio real de notificações** por email ou SMS
 - ☒ **Relatórios e estatísticas** avançadas
 - ☒ **Integração com calendários** externos
 - ☒ **Sistema de permissões** granulares
 - ☒ **Histórico de alterações** (Audit Log)
 - ☒ **Sincronização** com sistemas externos
 - ☒ **Implementação de Repositório** (apenas interface)
-

Usuários do Sistema

Usuários Diretos

1. Usuário Final (Pessoa Física)

- **Quem é:** Qualquer pessoa que precisa gerenciar suas tarefas
- **O que faz:**
 - Cria novas tarefas
 - Visualiza tarefas existentes
 - Marca tarefas como concluídas
 - Atualiza informações de tarefas

- Exclui tarefas
 - **Exemplo:** "João cria uma tarefa 'Estudar DDD' com prazo em 7 dias e prioridade Alta"
-

Interfaces com Outros Sistemas

2. Sistema de Notificações (Interface Externa)

- **Quem é:** Serviço externo que envia alertas
- **O que recebe:**
 - ID da tarefa
 - Título da tarefa
 - Data de vencimento
- **Como integra:**
 - Via **Anti-Corruption Layer** (TarefaAdapter)
 - Recebe DTO traduzido (TarefaVencendoDTO)
 - Não conhece detalhes complexos da Tarefa original
- **Exemplo:** "Sistema de Notificações recebe alerta: 'Estudar DDD vence em 2 dias'"

3. Banco de Dados (Interface de Persistência)

- **Quem é:** Sistema de armazenamento de dados
 - **O que faz:**
 - Persiste tarefas
 - Recupera tarefas por ID
 - Atualiza tarefas
 - Deleta tarefas
 - **Como integra:**
 - Via **Repository Pattern** (ITarefaRepository)
 - Interface define contrato, implementação é agnóstica (SQL, MongoDB, etc)
 - **Exemplo:** "Repository salva Tarefa no banco de dados"
-

Ubiquitous Language

Termos Principais do Domínio

Termo	Definição	Contexto	Exemplo
Tarefa	Unidade de trabalho que precisa ser realizada	Tarefas	"Criar uma tarefa para estudar DDD"

Prazo	Data até quando a tarefa deve ser concluída	Tarefas	"Prazo de 7 dias a partir de hoje"
Prioridade	Nível de importância da tarefa	Tarefas	"Tarefa com prioridade Alta"
Concluir	Marcar uma tarefa como finalizada	Tarefas	"Concluir a tarefa de estudar"
Usuário	Pessoa que cria e gerencia tarefas	Tarefas	"O usuário João criou a tarefa"
Tarefa Vencendo	Tarefa próxima do prazo (dentro de 3 dias)	Notificações	"A tarefa 'Estudar' está vencendo"
Notificar	Alertar sobre tarefa próxima do vencimento	Notificações	"Notificar o usuário sobre a tarefa vencendo"

Frases Comuns no Domínio

- **"Criar uma tarefa"** → Instanciar nova Tarefa via TarefaFactory
- **"Concluir uma tarefa"** → Chamar ServicoDeConclusao.ConcluirTarefa()
- **"Validar prazo"** → Verificar se data é futura no construtor de Prazo
- **"Tarefa vencendo"** → Tarefa com prazo próximo (< 3 dias)
- **"Integração entre contextos"** → TarefaAdapter traduz Tarefa para TarefaVencendoDTO

Regras de Negócio

Regra 1: Título da Tarefa é Obrigatório

Descrição: Uma tarefa não pode ser criada sem um título.

Detalhes:

- Campo: Titulo (string)
- Validação: Não pode ser nulo, vazio ou apenas espaços em branco
- Ação se violada: Lança ArgumentException

Exemplo:

```
// ❌ Falha
var tarefa = new Tarefa("", "Descrição", usuarioId, prazo, prioridade);
// ArgumentException: "O título da tarefa é obrigatório."

// ✅ Passa
```

```
var tarefa = new Tarefa("Estudar DDD", "Descrição", usuarioId, prazo,
prioridade);
```

Regra 2: Prazo não pode ser no Passado

Descrição: Uma tarefa não pode ter prazo em uma data anterior à data atual.

Detalhes:

- Campo: Prazo.Data (DateTime)
- Validação: Deve ser sempre uma data futura (> DateTime.UtcNow)
- Ação se violada: Lança ArgumentException

Exemplo:

```
// ❌ Falha
var prazo = new Prazo(DateTime.UtcNow.AddDays(-7));
// ArgumentException: "O prazo da tarefa deve ser uma data futura."

// ✅ Passa
var prazo = new Prazo(DateTime.UtcNow.AddDays(7));
```

Regra 3: Tarefa Concluída não pode ser Alterada

Descrição: Uma tarefa que já foi marcada como concluída não pode ser concluída novamente.

Detalhes:

- Campo: Concluida (bool)
- Validação: Se já é true, não pode chamar Concluir() novamente
- Ação se violada: Lança InvalidOperationException

Exemplo:

```
// ✅ Primeira conclusão passa
await servicoDeConclusao.ConcluirTarefa(tarefaId);
// Tarefa.Concluida = true

// ❌ Segunda conclusão falha
```

```
await servicoDeConclusao.ConcluirTarefa(tarefaId);  
// InvalidOperationException: "A tarefa já está concluída."
```

Testes Unitários

A aplicação inclui **8 testes unitários** cobrindo todos os cenários de negócio:

Testes de Prazo.cs

Teste	Objetivo	Status
CriarPrazo_ComDataValida_DeveCriarComSucesso	Validar que prazo com data futura é criado	✓
CriarPrazo_ComDataPassada_DeveLancarExcecao	Validar que prazo no passado lança exception	✓

Testes de Tarefa.cs

Teste	Objetivo	Status
CriarTarefa_ComTituloValido_DeveCriarTarefa	Validar que tarefa com título válido é criada	✓
CriarTarefa_ComTituloInvalido_DeveLancarExcecao	Validar que título vazio lança exception	✓
CriarTarefa_ComPrazoPassado_DeveLancarExcecao	Validar que prazo inválido lança exception	✓
CriarTarefa_ComUsuarioVazio_DeveLancarExcecao	Validar que usuário ID vazio lança exception	✓

Testes de servicoDeConclusao.cs

Teste	Objetivo	Status
ConcluirTarefa_ComTarefaValida_DeveConcluirComSucessoAsync	Validar que tarefa válida é concluída	✓
ConcluirTarefa_ComTarefaJaConcluida_DeveLancarExcecao	Validar que reconclusão lança exception	✓

Diagrama da Arquitetura DDD

```

graph TB
    subgraph Tarefas["CONTEXTO: TAREFAS"]
        Entity["<b>Entity</b><br/>——<br/>■ Id: Guid<br/>——<br/>+ Equals()  
<br/>+ GetHashCode()"]

        Tarefa["<b>Tarefa</b><br/><i>(Aggregate Root)</i><br/>——<br/>■  
Titulo: string<br/>■ Descricao: string<br/>■ Concluida: bool<br/>■ UsuarioId:  
Guid<br/>■ Prazo: Prazo<br/>■ Prioridade: Prioridade<br/>——<br/>+ Validar()  
<br/>+ Concluir()"]

        Usuario["<b>Usuario</b><br/><i>(Entity)</i><br/>——<br/>■ Nome:  
string<br/>——<br/>+ Validar()"]

        Prazo["<b>Prazo</b><br/><i>(Value Object)</i><br/>——<br/>■ Data:  
DateTime<br/>——<br/>+ Validar()<br/>+ Equals()"]

        Prioridade["<b>Prioridade</b><br/><i>(Value Object)</i><br/>——<br/>■  
Nivel: enum<br/>——<br/>+ Equals()"]

        Factory["<b>TarefaFactory</b><br/><i>(Factory)</i><br/>——<br/>+  
CriarTarefa()"]

        Service["<b>ServicoDeConclusao</b><br/><i>(Domain Service)</i>  
<br/>——<br/>+ ConcluirTarefa()"]

        Repository["<b>ITarefaRepository</b><br/><i>interface</i>  
<br/>——<br/>+ ObterPorId()<br/>+ Adicionar()<br/>+ Atualizar()<br/>+  
Remover()"]

        Entity -->|herança| Tarefa
        Entity -->|herança| Usuario
        Entity -->|herança| Prazo
        Entity -->|herança| Prioridade

        Tarefa -->|contém| Prazo
        Tarefa -->|contém| Prioridade

        Factory -->|cria| Tarefa
        Service -->|usa| Repository
        Service -->|opera| Tarefa
    end

    subgraph Notificacoes["CONTEXTO: NOTIFICAÇÕES"]
        Adapter["<b>TarefaAdapter</b><br/><i>(Anti-Corruption Layer)</i>  
<br/>——<br/>+ AdaptarTarefa()"]
    end

```



```

    DTO["<b>TarefaVencendoDT0</b><br><i>(DT0)</i><br>——<br>■ Id:
    Guid<br>■ Titulo: string<br>■ DataVencimento: DateTime"]

```

```

    Adapter -->|traduz em| DTO
end

```

```

Tarefa -.->|ACL| Adapter

```

```

style Entity fill:#fff3e0,stroke:#f57c00,stroke-width:2px
style Tarefa fill:#fff3e0,stroke:#f57c00,stroke-width:2px
style Usuario fill:#fff3e0,stroke:#f57c00,stroke-width:2px
style Prazo fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px
style Prioridade fill:#f3e5f5,stroke:#7b1fa2,stroke-width:2px
style Factory fill:#e8f5e9,stroke:#388e3c,stroke-width:2px
style Service fill:#fce4ec,stroke:#c2185b,stroke-width:2px
style Repository fill:#fff9c4,stroke:#f9a825,stroke-width:2px
style Adapter fill:#ffebee,stroke:#d32f2f,stroke-width:2px
style DTO fill:#ffe0b2,stroke:#e65100,stroke-width:2px
style Tarefas fill:#e3f2fd,stroke:#1976d2,stroke-width:2px
style Notificacoes fill:#e8f5e9,stroke:#388e3c,stroke-width:2px

```

Legenda de Componentes

Cor	Componente	Descrição
 Laranja	Entities	Objetos com identidade única (Id). Comparados pelo Id.
 Roxo	Value Objects	Objetos imutáveis comparados por valor (Prazo, Prioridade).
 Verde	Factories	Padrão criador. Responsabilidade: CRIAR agregados.
 Rosa	Domain Services	Padrão orquestrador. Responsabilidade: OPERAR sobre agregados.
 Amarelo	Repositories	Padrão persistência. Abstrai como dados são salvos.
 Vermelho	Anti-Corruption Layer	Traduz conceitos entre Bounded Contexts (desacoplamento).
 Laranja claro	DTOs	Objetos de transferência de dados entre contextos.
 Azul	Bounded Contexts	Limites de domínio isolados e independentes.

Relações Principais

- **Herança:** Tarefa, Usuario, Prazo e Prioridade herdam de Entity
- **Composição:** Tarefa contém Prazo e Prioridade (Value Objects)
- **Criação:** TarefaFactory cria novas instâncias de Tarefa
- **Persistência:** ServicoDeConclusao usa ITarefaRepository

- **Desacoplamento:** TarefaAdapter traduz Tarefa em TarefaVencendoDTO (ACL)

Critérios de Avaliação

1. Orientação a Objetos com C#

#	Critério	Descrição	Link do Código
1.1	OO Básico	O aluno implementou as classes aplicando os conceitos básicos de OO como Encapsulamento, Abstração, Herança e Polimorfismo?	Tarefa (classe de domínio)
1.2	Modificadores de Acesso	O aluno implementou as classes e objetos em C#, aplicando corretamente modificadores de acesso, propriedades, métodos e construtores?	Propriedade com private set
1.3	Herança e Polimorfismo	O aluno aplicou herança e polimorfismo em C# para criar hierarquias de classes flexíveis e extensíveis?	Herança em Tarefa : Entity
1.4	Abstração e Encapsulamento	O aluno aplicou abstração e encapsulamento em C# para ocultar detalhes de implementação e expor interfaces claras e concisas?	Entity abstrata

2. Domain-Driven Design

#	Critério	Descrição	Link do Código
2.1	UL, Entities, VOs, Repositories	O aluno modelou o domínio utilizando Ubiquitous Language, Entities, Value Objects e Repositories de forma coerente com os conceitos de DDD?	Entity · Value Object · Repository
2.2	Aggregates, Bounded Contexts, Services	O aluno modelou o domínio utilizando Aggregate, Bounded Contexts e Domain Services de maneira estruturada e adequada ao problema?	Aggregate (Tarefa) · Domain Service
2.3	Domain Services vs Factories	O aluno diferenciou claramente Domain Services e Factories na modelagem do domínio, justificando sua escolha com base na responsabilidade de cada elemento?	Factory · Service
2.4	ACL e Context Map	O aluno modelou o domínio considerando a integração entre Bounded Contexts, aplicando padrões como Anti-Corruption Layer e Context Map com clareza e eficácia?	Anti-Corruption Layer (TarefaAdapter)

3. Padrões de Projeto - SOLID e GRASP

#	Critério	Descrição	Link do Código
3.1	Princípios SOLID	O aluno aplicou os princípios SOLID no design das classes, garantindo coesão, alta responsabilidade e baixo acoplamento?	DIP: serviço depende de abstração
3.2	Single Responsibility	O aluno utilizou corretamente o princípio de Single Responsibility nas classes, evitando a concentração excessiva de responsabilidades?	Factory com responsabilidade única de criação
3.3	Low Coupling	O aluno aplicou o padrão Low Coupling para garantir a independência entre as classes e promover a reutilização de código?	Acoplamento baixo via interface no construtor
3.4	Controller (GRASP)	O aluno utilizou o padrão Controller de forma adequada, promovendo a separação entre lógica de controle e demais responsabilidades?	Orquestração do caso de uso em ServicoDeConclusao

4. Testes Unitários e TDD

#	Critério	Descrição	Link do Código
4.1	Princípios de Testes	O aluno aplicou corretamente os princípios de testes unitários como isolamento, repetibilidade, rapidez, auto-verificação e abrangência?	Teste unitário isolado de Value Object
4.2	Cobertura de Métodos	O aluno implementou testes unitários abrangendo todos os métodos que contêm regras de negócio relevantes?	Cenários de criação/validação em TarefaTests
4.3	Mocks e Stubs	O aluno utilizou mocks e stubs de maneira adequada para isolar o código sob teste durante a implementação dos testes unitários?	Mock do repositório com Moq
4.4	Cobertura >80%	O aluno implementou testes unitários com cobertura superior a 80% do código de domínio, garantindo qualidade e confiabilidade da aplicação?	Suíte de testes de domínio